

0.1 Đặt vấn đề

Trong bối cảnh các hoạt động trao đổi thông tin nội bộ ngày càng phụ thuộc vào hạ tầng số, bảo vệ nội dung liên lạc trở thành một yêu cầu quan trọng đối với tổ chức, doanh nghiệp và cơ quan quản lý nhà nước. Rò rỉ thông tin nhạy cảm, tài liệu nghiệp vụ hoặc nội dung trao đổi nội bộ có thể gây thiệt hại trực tiếp về kinh tế, uy tín và an toàn vận hành. Một trong những hướng tiếp cận quan trọng để giảm rủi ro này là mã hóa đầu cuối (End-to-End Encryption – E2EE), trong đó nội dung thông điệp chỉ được giải mã tại các thiết bị đầu cuối hợp lệ, còn các thành phần trung gian trong quá trình truyền tải không được quyền truy cập bản rõ của thông điệp **rfc9420, rfc9750**.

Trong nhiều hệ thống nhắn tin bảo mật hiện nay, máy chủ vẫn giữ vai trò quan trọng trong việc định tuyến tin nhắn, lưu trữ khóa công khai ban đầu, hỗ trợ thiết bị ngoại tuyến và đồng bộ trạng thái giữa các thành viên. Cách tổ chức này không đồng nghĩa với việc nội dung tin nhắn luôn bị lộ cho máy chủ, nhưng nó vẫn tạo ra sự phụ thuộc đáng kể vào hạ tầng vận hành trung tâm. Nếu hạ tầng này gặp sự cố, bị tấn công từ chối dịch vụ, bị cấu hình sai hoặc bị kiểm soát bởi một tác nhân không mong muốn, khả năng liên lạc của toàn hệ thống có thể bị ảnh hưởng. Ngoài ra, ngay cả khi nội dung được mã hóa, các thành phần trung gian vẫn có thể quan sát một số loại siêu dữ liệu như thời điểm gửi, tần suất liên lạc, kích thước thông điệp, thông tin định tuyến hoặc quan hệ giữa các thiết bị, tùy theo thiết kế của từng hệ thống **rfc9750**.

Với các môi trường đặc thù như mạng nội bộ biệt lập, mạng tạm thời không có Internet hoặc tổ chức cần kiểm soát chặt chẽ dữ liệu, sự phụ thuộc vào một máy chủ Internet trung tâm trở thành hạn chế thực tế. Các hệ thống này cần một mô hình trong đó dữ liệu ưu tiên được lưu trữ cục bộ, thiết bị có thể tiếp tục hoạt động khi mất kết nối với máy chủ bên ngoài, và các nút mạng có khả năng trao đổi trực tiếp khi cùng xuất hiện trong mạng LAN, VPN hoặc một mạng ngang hàng. Định hướng này gần với mô hình Local-First và Peer-to-Peer (P2P): dữ liệu và trạng thái ứng dụng được ưu tiên giữ tại thiết bị của người dùng, còn đồng bộ chỉ diễn ra khi các thiết bị có điều kiện kết nối.

Tuy nhiên, xây dựng một hệ thống liên lạc nhóm vừa mã hóa đầu cuối, vừa hoạt động trên mạng ngang hàng bất đồng bộ là một bài toán khó. Với liên lạc hai bên, Double Ratchet đã trở thành một cơ chế phổ biến để tạo khóa mới liên tục cho từng thông điệp, cung cấp tính bảo mật xuôi và khả năng phục hồi sau thỏa hiệp ở mức phiên hai bên **perrin2025doubleratchet**. Khi chuyển sang nhóm có nhiều thành viên, bài toán không chỉ là mã hóa từng tin nhắn, mà còn là duy trì một trạng thái

mật mã nhóm nhất quán giữa nhiều thiết bị có thể online, offline, gửi tin không đồng thời và nhận thông điệp theo các thứ tự khác nhau.

Messaging Layer Security (MLS), được chuẩn hóa trong RFC 9420, được thiết kế để giải quyết bài toán thiết lập khóa nhóm bất đồng bộ với các bảo đảm quan trọng như Forward Secrecy (FS) và Post-Compromise Security (PCS). MLS sử dụng cấu trúc cây mật mã để làm cho chi phí cập nhật khóa tăng theo logarit kích thước nhóm, thay vì phải phân phối khóa lại theo cách tuyến tính hoặc bình phương trong nhiều kịch bản nhóm lớn **rfc9420**. Tuy nhiên, MLS không phải là một giao thức nhắn tin hoàn chỉnh. Nó cần được tích hợp vào một hệ thống nhắn tin cụ thể, trong đó phần ứng dụng phải quyết định cách định tuyến thông điệp, cách xác thực danh tính, cách lưu trữ vật liệu khóa ban đầu và đặc biệt là cách xử lý thứ tự các thông điệp thay đổi trạng thái nhóm **rfc9750**.

Chính tại đây xuất hiện vấn đề trung tâm của đề tài. Trong các triển khai có Delivery Service nhất quán mạnh, dịch vụ phân phối có thể đóng vai trò sắp xếp thứ tự Commit và giải quyết xung đột khi nhiều thành viên cùng cập nhật nhóm tại một epoch. Trong môi trường P2P không có một máy chủ trung tâm điều phối, các Commit có thể được phát tán qua mạng theo các thứ tự khác nhau, khiến các thiết bị khác nhau áp dụng những thứ tự thay đổi trạng thái nhóm khác nhau, dẫn đến các trạng thái khác nhau. Nếu không có cơ chế điều phối nhất quán cho các thiết bị, nhóm MLS có thể rơi vào trạng thái phân nhánh mật mã, làm cho các thành viên không còn chia sẻ cùng một trạng thái khóa nhóm, dẫn đến không thể mã hóa và giải mã tin nhắn của nhau trong nhóm.

Từ bối cảnh đó, đề tài “Nghiên cứu và ứng dụng giao thức điều phối phi tập trung cho MLS trên mạng ngang hàng” tập trung vào bài toán thiết kế một lớp điều phối ở tầng ứng dụng để hỗ trợ vận hành MLS trong môi trường P2P. Mục tiêu không phải thay đổi lõi mật mã của MLS, mà là bổ sung cơ chế điều phối, kiểm tra epoch, xử lý xung đột và phục hồi sau phân mảnh mạng để MLS có thể được sử dụng an toàn hơn trong một hệ thống liên lạc nhóm phi tập trung.

0.2 Các giải pháp hiện tại và hạn chế

Các hướng tiếp cận hiện tại cho mã hóa nhóm có thể được chia thành hai nhóm chính. Nhóm thứ nhất dựa trên các kênh bảo mật hai bên, sau đó mở rộng sang nhóm bằng cách phân phối khóa gửi tin cho từng thành viên. Nhóm thứ hai dựa trên giao thức thiết lập khóa nhóm chuyên biệt, tiêu biểu là MLS và TreeKEM. Mỗi hướng đều có ưu điểm riêng, nhưng cũng để lại các giới hạn quan trọng khi đặt trong yêu cầu vận hành P2P hoàn toàn phi tập trung.

0.2.1 Double Ratchet, Sender Keys và Megolm

Double Ratchet là thuật toán trao đổi thông điệp mã hóa giữa hai bên dựa trên một bí mật chung đã được thiết lập từ trước. Sau giai đoạn khởi tạo, hai bên liên tục dẫn xuất khóa mới cho từng thông điệp thông qua sự kết hợp giữa ratchet đối xứng và ratchet Diffie–Hellman. Cơ chế ratchet đối xứng giúp mỗi thông điệp sử dụng một khóa riêng biệt và có thể xóa sau khi dùng, trong khi ratchet Diffie–Hellman đưa thêm entropy mới vào phiên liên lạc khi hai bên trao đổi khóa công khai mới. Nhờ đó, Double Ratchet cung cấp khả năng bảo vệ thông điệp quá khứ và khôi phục bảo mật cho thông điệp tương lai trong một số kịch bản thỏa hiệp khóa **perrin2025doubleratchet**. Đây là nền tảng quan trọng của nhiều giao thức nhắn tin bảo mật hiện đại.

Khi mở rộng từ liên lạc hai bên sang liên lạc nhóm, một hướng tiếp cận phổ biến là sử dụng cơ chế Sender Keys. Trong mô hình này, mỗi thành viên tự sinh một khóa gửi tin riêng và phân phối khóa đó đến các thành viên còn lại thông qua các kênh bảo mật hai bên. Sau khi khóa gửi tin đã được phân phối, các thông điệp tiếp theo của cùng người gửi có thể được mã hóa thành một bản mã duy nhất, sau đó được máy chủ hoặc lớp mạng phân phối tới các thành viên trong nhóm. Vì vậy, Sender Keys cải thiện đáng kể chi phí gửi tin so với mô hình mã hóa riêng biệt cho từng người nhận trong mỗi lần gửi thông điệp **rfc9420, whatsapp2026encryption**.

Tuy nhiên, giới hạn chính của Sender Keys nằm ở khả năng đạt bảo mật sau thỏa hiệp và chi phí quản lý thay đổi thành viên. Khi khóa gửi tin của một thành viên bị lộ, kẻ tấn công có thể tiếp tục giải mã thụ động các thông điệp do thành viên đó gửi cho đến khi khóa mới được sinh và phân phối lại. Việc tạo và phân phối lại khóa gửi tin cho một thành viên có chi phí tăng tuyến tính theo kích thước nhóm; nếu nhiều thành viên cùng cần cập nhật khóa, tổng chi phí điều phối và truyền thông có thể tăng đáng kể **rfc9420**. Do đó, Sender Keys phù hợp với nhiều hệ thống nhắn tin thực tế, nhưng chưa phải lời giải tối ưu cho các nhóm lớn có biến động thành viên thường xuyên hoặc yêu cầu Post-Compromise Security mạnh.

Megolm là cơ chế mã hóa nhóm được sử dụng trong hệ sinh thái Matrix, với mục tiêu chính là tối ưu chi phí gửi tin trong các phòng trò chuyện nhiều thành viên. Thay vì thiết lập một kênh mã hóa riêng cho từng cặp thiết bị, Megolm sử dụng mô hình *group ratchet*: một sender tạo ra một outbound session và dùng session này để mã hóa chuỗi thông điệp gửi vào phòng **matrix2026megolm**. Cách thiết kế này giúp việc gửi tin trong nhóm trở nên nhẹ hơn đáng kể, vì mỗi thông điệp không cần được mã hóa lại riêng biệt cho từng người nhận.

Đánh đổi của Megolm nằm ở chỗ session state có vòng đời dài hơn so với các

cơ chế ratchet chặt chẽ, vốn được gắn theo từng tin nhắn. Khi ratchet state của một Megolm session bị lộ tại một thời điểm, kẻ tấn công có thể suy ra các tin nhắn từ thời điểm đó trở về sau trong cùng session. Vì vậy, Megolm không cung cấp PCS mạnh như mong muốn đối với các nhóm có yêu cầu bảo mật cao; đồng thời, đặc tả của Matrix cũng ghi nhận các giới hạn liên quan đến transcript consistency và forward secrecy **matrix2026megolm**. Trong thực tế, các hệ thống dùng Megolm cần định kỳ tạo session mới và phân phối lại session key để thu hẹp khoảng thời gian bị ảnh hưởng nếu thiết bị hoặc session state bị compromise.

Như vậy, Double Ratchet, Sender Keys và Megolm đều là các thiết kế có giá trị thực tiễn lớn và đã chứng minh hiệu quả trong nhiều hệ thống nhắn tin bảo mật. Tuy nhiên, khi bài toán chuyển sang nhóm lớn, membership thay đổi thường xuyên và yêu cầu PCS mạnh hơn, các cơ chế này bắt đầu bộc lộ giới hạn ở khâu rekey, quản lý session và phục hồi sau compromise. Đây là khoảng trống mà Messaging Layer Security (MLS) hướng tới giải quyết bằng cách đưa việc quản lý khóa nhóm về một cấu trúc có hệ thống hơn, trong đó TreeKEM cho phép cập nhật trạng thái nhóm với chi phí hiệu quả hơn và phù hợp hơn cho truyền thông nhóm quy mô lớn.

0.2.2 MLS và bài toán Delivery Service trong triển khai phân tán

MLS được chuẩn hóa nhằm cung cấp cơ chế thiết lập khóa nhóm bất đồng bộ, hiệu quả và có bảo đảm FS/PCS cho các nhóm từ quy mô nhỏ đến lớn. Thay vì dựa vào việc mỗi thành viên phân phối khóa gửi tin riêng lẻ cho toàn bộ nhóm, MLS biểu diễn trạng thái nhóm bằng một cây mật mã. Khi thành viên thay đổi khóa hoặc khi nhóm thay đổi thành viên, thao tác cập nhật chỉ cần tác động đến một đường đi trong cây, làm cho chi phí cập nhật tăng theo logarit kích thước nhóm trong mô hình lý thuyết của TreeKEM **rfc9420**.

Dù vậy, RFC 9750 nhấn mạnh rằng MLS không phải là một giao thức nhắn tin hoàn chỉnh, mà là một giao thức mật mã cần được nhúng vào hệ thống ứng dụng cụ thể **rfc9750**. Một hệ thống sử dụng MLS thường cần hai chức năng trừu tượng: Authentication Service để ràng buộc danh tính ứng dụng với khóa xác thực, và Delivery Service để lưu trữ vật liệu khóa ban đầu, nhận và phân phối thông điệp giữa các thành viên **rfc9750**. Quan trọng hơn, RFC 9750 không bắt buộc Delivery Service phải là một máy chủ trung tâm duy nhất. Delivery Service có thể được triển khai trong mô hình một nhà cung cấp, liên miền, hoặc thậm chí được hiện thực hóa phi tập trung bằng cách truyền thông điệp MLS qua mạng ngang hàng **rfc9750**.

Điểm khó nhất của việc triển khai MLS trong mạng ngang hàng nằm ở tính nhất quán của các thông điệp thay đổi trạng thái nhóm. Trong MLS, Commit là thông điệp khởi tạo một epoch mới bằng cách yêu cầu các thành viên áp dụng một tập

thay đổi như thêm thành viên, loại bỏ thành viên hoặc cập nhật khóa **rfc9750**. Nếu hai thành viên cùng tạo Commit dựa trên cùng một epoch, hệ thống phải có cách quyết định Commit nào được chấp nhận trước, hoặc có cách hòa giải khi các thành viên nhìn thấy các Commit theo thứ tự khác nhau. Trong hệ thống sử dụng Delivery Service nhất quán mạnh, nó có thể phá thống nhất thứ tự bằng cách áp đặt một thứ tự tuyến tính cho các Commit. Trong các hệ thống Delivery Service theo triết lý nhất quán cuối cùng hoặc trong mạng P2P, thông điệp có thể đến các thiết bị theo thứ tự khác nhau, và khi đó client phải tham gia vào quá trình hòa giải **rfc9750**.

Một hướng nghiên cứu gần với bài toán này là Decentralized MLS (DMLS) của Kohbrok. Thay vì giữ nguyên MLS và giải quyết xung đột ở tầng hệ thống, DMLS chọn cách mở rộng trực tiếp bản thân giao thức MLS để hỗ trợ xử lý các Commit đến ngoài thứ tự trong môi trường không có máy chủ trung tâm áp đặt thứ tự. Hướng tiếp cận này cho phép client giữ lại một phần tài nguyên key mã hóa để xử lý các nhánh Commit khác nhau với mức ảnh hưởng tới Forward Secrecy thấp hơn so với việc giữ khóa cũ một cách thô sơ **alwen2023fork**, **kohbrok2023dmls**. Tuy nhiên, đặc trưng này cũng là điểm yếu chính của DMLS: để xử lý out-of-order commits, client vẫn phải duy trì thêm trạng thái mật mã cũ hoặc trạng thái tạm thời, làm tăng độ phức tạp triển khai. Quan trọng hơn, DMLS hiện vẫn chỉ là một Internet-Draft đã hết hạn, chưa phải là một chuẩn IETF hoàn chỉnh **kohbrok2023dmls**. Vì vậy, đề án này lựa chọn hướng thực dụng hơn: giữ nguyên lõi MLS và giải quyết bài toán ordering, fork detection và fork healing ở tầng điều phối phía ngoài tầng MLS.

0.3 Mục tiêu và định hướng giải pháp

0.3.1 Mục tiêu của đề án

Mục tiêu của đề án là nghiên cứu, thiết kế và hiện thực một lớp điều phối phi tập trung ở tầng ứng dụng nhằm hỗ trợ vận hành Messaging Layer Security (MLS) trên mạng ngang hàng. Lớp điều phối này không thay đổi lõi mật mã của MLS, mà đóng vai trò như một lớp bao quanh, chịu trách nhiệm xử lý các vấn đề phát sinh khi hệ thống không còn phụ thuộc vào một Delivery Service trung tâm có khả năng áp đặt thứ tự Commit một cách nhất quán.

Về mặt lý thuyết, đề án tập trung vào ba mục tiêu chính. Thứ nhất, đề án mô hình hóa bài toán xung đột Commit trong MLS khi các thông điệp handshake được truyền qua môi trường P2P bất đồng bộ, nơi thông điệp có thể đến trễ, đến sai thứ tự hoặc bị chia cắt bởi network partition. Thứ hai, đề án đề xuất cơ chế điều phối dựa trên nguyên tắc *Single-Writer per Epoch*, trong đó tại mỗi epoch chỉ một nút được quyền tạo Commit, qua đó giảm đáng kể khả năng xuất hiện concurrent commits trong điều kiện mạng liên thông. Thứ ba, đề án xây dựng cơ chế phát hiện và phục

hồi khi phân nhánh trạng thái nhóm vẫn xảy ra, đặc biệt trong các tình huống mạng bị phân mảnh vật lý rồi kết nối trở lại.

Về mặt ứng dụng, đồ án hướng tới xây dựng một ứng dụng cộng tác nội bộ bảo mật, hỗ trợ các chức năng cốt lõi như chat nhóm, lưu trữ dữ liệu cục bộ, đồng bộ trạng thái giữa các thiết bị, chia sẻ tệp tin và phát hiện peer trong mạng ngang hàng. Ứng dụng được thiết kế theo định hướng *Local-First* và giảm phụ thuộc vào hạ tầng máy chủ trung tâm, nhờ đó có thể vận hành trong các kịch bản mạng nội bộ, mạng tạm thời mất Internet hoặc môi trường cần kiểm soát dữ liệu chặt chẽ. Nguyên mẫu này đóng vai trò kiểm chứng tính khả thi của giao thức điều phối đề xuất, đồng thời cung cấp cơ sở thực nghiệm để đánh giá độ trễ, chi phí truyền thông, khả năng phục hồi sau mất kết nối và giới hạn mở rộng của hệ thống.

Trong môi trường phân tán bất đồng bộ, đặc biệt khi xảy ra *network partition*, việc đồng thời bảo đảm tính nhất quán mạnh, tính sẵn sàng và khả năng chịu phân mảnh là không thể. Theo định lý CAP, khi các nút trong hệ phân tán bị chia cắt bởi lỗi mạng, hệ thống buộc phải đánh đổi giữa *Consistency* và *Availability* nếu vẫn muốn duy trì *Partition Tolerance* **gilbert2002brewers**. Đối với bài toán của đồ án, *Partition Tolerance* là yêu cầu gần như bắt buộc vì hệ thống được thiết kế cho môi trường P2P, nơi thiết bị có thể offline, độ trễ mạng không ổn định và các phân vùng mạng có thể xuất hiện bất cứ lúc nào.

Từ góc nhìn này, đồ án lựa chọn hướng thiết kế thiên về *AP-oriented*: ưu tiên để các phân vùng mạng vẫn có khả năng hoạt động cục bộ trong thời gian bị chia cắt, thay vì dừng toàn bộ hệ thống để chờ một cơ chế đồng thuận toàn cục. Các cơ chế đồng thuận mạnh như BFT, blockchain hoặc quorum-based consensus có thể duy trì một thứ tự Commit duy nhất, nhưng thường đánh đổi bằng độ trễ lớn, chi phí truyền thông cao và mất tính sẵn sàng ở các phân vùng không đạt quorum. Điều này không phù hợp với mục tiêu *Local-First* và P2P của đồ án.

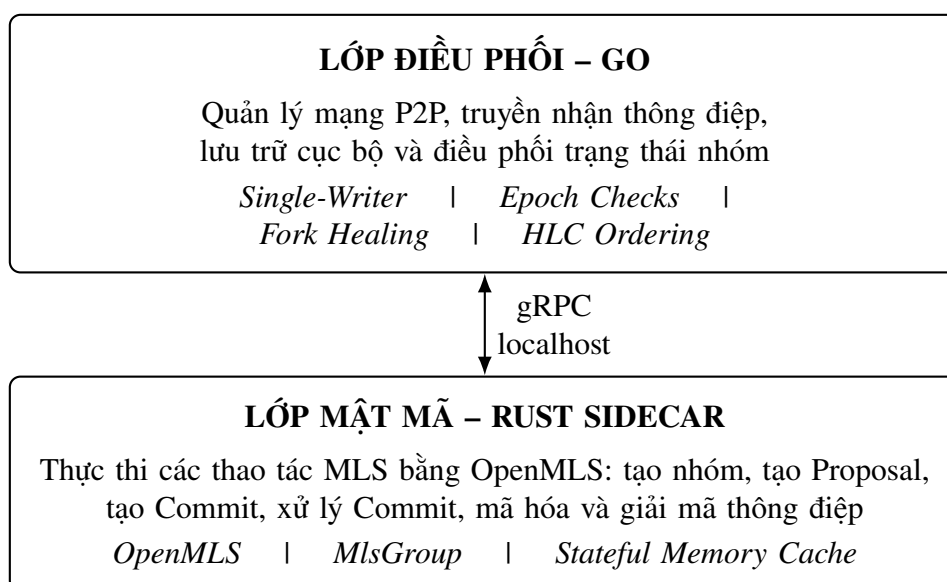
Tuy nhiên, do trạng thái MLS là trạng thái mật mã chứ không phải dữ liệu ứng dụng có thể hợp nhất tùy ý, hệ thống không thể chấp nhận phân kỳ vô hạn. Vì vậy, giải pháp đề xuất không theo hướng AP thuần túy, mà kết hợp giữa tính sẵn sàng cục bộ và cơ chế hội tụ có kiểm soát. Cụ thể, giao thức hạn chế concurrent commits trong điều kiện mạng liên thông bằng Single-Writer, phát hiện phân nhánh trạng thái khi partition xảy ra, và sử dụng Fork Healing để đưa các phân vùng hội tụ trở lại một trạng thái MLS hợp lệ duy nhất sau khi mạng được kết nối lại.

0.3.2 Định hướng giải pháp

Từ các mục tiêu đã nêu, đồ án lựa chọn hướng tiếp cận xây dựng một lớp điều phối phi tập trung đặt phía trên lõi mật mã MLS. Lớp này không thay đổi các thuật

toán mật mã của MLS, mà bổ sung các cơ chế cần thiết để MLS có thể vận hành trong môi trường mạng ngang hàng, nơi không còn một Delivery Service trung tâm đảm nhiệm vai trò phân phối và sắp xếp thứ tự Commit.

Kiến trúc hệ thống được chia thành hai lớp chính, như minh họa trong Hình 0.1. Lớp thứ nhất là *Coordination Layer*, được hiện thực bằng Go. Lớp này chịu trách nhiệm quản lý mạng P2P, truyền nhận thông điệp qua libp2p/GossipSub, duy trì trạng thái ứng dụng cục bộ, điều phối quá trình chuyển epoch, phát hiện xung đột và xử lý các tình huống mất đồng bộ. Lớp thứ hai là *Crypto Layer*, được hiện thực bằng Rust trên cơ sở thư viện OpenMLS. Lớp này đảm nhiệm toàn bộ các thao tác mật mã của MLS như tạo nhóm, tạo Proposal, tạo Commit, xử lý Commit, mã hóa và giải mã thông điệp. Hai lớp giao tiếp với nhau thông qua gRPC trên localhost theo mô hình sidecar.



Hình 0.1: Kiến trúc hai lớp của hệ thống đề xuất

Trong kiến trúc này, Go đóng vai trò điều phối hệ thống, còn Rust đóng vai trò lõi mật mã tin cậy. Việc tách hai lớp giúp hệ thống tận dụng được ưu điểm của từng ngôn ngữ: Go phù hợp với xử lý đồng thời, lập trình mạng và tích hợp libp2p; Rust phù hợp với các thao tác mật mã yêu cầu an toàn bộ nhớ và hiệu năng cao. Đồng thời, cách tổ chức này giúp đề án tránh việc tự lập trình lại MLS, thay vào đó sử dụng thư viện OpenMLS như một thư viện chuẩn RFC 9420.

Lớp điều phối được xây dựng xung quanh bốn cơ chế chính. Cơ chế thứ nhất là *Single-Writer per Epoch*. Tại mỗi epoch, hệ thống xác định một thành viên duy nhất có quyền tạo Commit, gọi là *Token Holder*. Token Holder được chọn bằng một hàm tất định dựa trên định danh nhóm, số epoch và tập thành viên hợp lệ. Nhờ vậy, trong điều kiện các nút có cùng góc nhìn về trạng thái nhóm, toàn bộ hệ thống

có thể tự xác định cùng một nút chịu trách nhiệm Commit mà không cần thêm một vòng bỏ phiếu hay đồng thuận mạng.

Cơ chế thứ hai là *Epoch Consistency Checks*. Mỗi thông điệp MLS nhận được từ mạng đều được kiểm tra theo `group_id`, `epoch`, loại thông điệp và trạng thái cục bộ trước khi được chuyển xuống lớp mật mã. Các thông điệp thuộc epoch cũ, epoch tương lai hoặc có dấu hiệu không tương thích với trạng thái hiện tại sẽ được xử lý theo chính sách riêng, chẳng hạn loại bỏ, đưa vào bộ đệm tạm thời hoặc kích hoạt đồng bộ trạng thái. Cơ chế này đóng vai trò như lớp bảo vệ giữa môi trường P2P bất đồng bộ và máy trạng thái mật mã của MLS.

Cơ chế thứ ba là *Fork Detection and Healing*. Trong trường hợp mạng bị phân mảnh, các phân vùng khác nhau có thể tiến hóa độc lập và tạo ra các nhánh trạng thái MLS khác nhau. Khi kết nối được khôi phục, các nút trao đổi thông tin trạng thái như epoch, tree hash, epoch authenticator và commit hash để phát hiện phân mảnh. Sau đó, hệ thống sử dụng một chính sách tất định để chọn nhánh tiếp tục, còn các nút ở nhánh không được chọn sẽ dừng sử dụng trạng thái cũ và gia nhập lại nhánh hợp lệ thông qua cơ chế phù hợp của MLS như Welcome hoặc External Commit **rfc9420**, **rfc9750**. Các thông điệp ứng dụng phát sinh trong thời gian phân mảnh, nếu cần phục hồi, sẽ được mã hóa lại và gửi lại dưới trạng thái MLS mới thay vì phát lại nguyên trạng bản mã cũ.

Cơ chế thứ tư là *HLC Message Ordering*. Commit trong MLS chỉ sắp xếp các thay đổi trạng thái mật mã, trong khi nhiều thông điệp ứng dụng vẫn có thể được gửi song song trong cùng một epoch. Do các thiết bị trong mạng P2P không thể giả định có đồng hồ vật lý đồng bộ tuyệt đối, hệ thống sử dụng Hybrid Logical Clock để gắn nhãn thời gian cho thông điệp ứng dụng, từ đó tạo thứ tự hiển thị ổn định hơn giữa các thiết bị **kulkarni2014hlc**. Cơ chế này chỉ phục vụ lớp ứng dụng và không thay thế các kiểm tra mật mã, kiểm tra epoch hay quá trình xử lý Commit của MLS.

Về mặt triển khai, Rust sidecar duy trì trạng thái MLS đang hoạt động trong bộ nhớ thông qua một cơ chế *stateful memory cache*. Thay vì truyền toàn bộ trạng thái cây MLS qua gRPC trong mỗi lời gọi, Go Host chỉ gửi định danh nhóm và dữ liệu cần xử lý, còn Rust sidecar tra cứu đối tượng `MlsGroup` tương ứng trong bộ nhớ. Thiết kế này giúp giảm chi phí giao tiếp liên tiến trình và hạn chế việc khởi tạo lặp lại các cấu trúc mật mã lớn. Trạng thái bền vững của ứng dụng vẫn được lưu trong SQLite phía Go Host để phục vụ khôi phục sau khi ứng dụng khởi động lại.

Tóm lại, định hướng giải pháp của đề án là giữ nguyên lõi mật mã MLS, đồng thời bổ sung một lớp điều phối phi tập trung ở tầng hệ thống để thay thế vai trò

tuần tự hóa thường do Delivery Service đảm nhiệm. Cách tiếp cận này cho phép hệ thống tận dụng các bảo đảm mật mã của MLS, đồng thời thích nghi với môi trường mạng ngang hàng bất đồng bộ và có khả năng phân mảnh.

0.4 Đóng góp của đề án

Từ mục tiêu và định hướng giải pháp đã trình bày, đề án mang lại hai nhóm đóng góp chính: đóng góp về mặt mô hình nghiên cứu và đóng góp về mặt hiện thực kỹ thuật.

0.4.1 Đóng góp về mặt nghiên cứu lý thuyết

Đóng góp đầu tiên của đề án là xây dựng cách tiếp cận mới cho bài toán vận hành MLS trong môi trường mạng ngang hàng. Thay vì xem Delivery Service như một thành phần máy chủ cố định, đề án phân tích vai trò của Delivery Service dưới dạng một tập các chức năng mà hệ thống triển khai MLS cần cung cấp, bao gồm phân phối thông điệp, hỗ trợ vật liệu khóa ban đầu, điều phối thứ tự Commit và xử lý khi trạng thái nhóm không còn đồng nhất. Cách tiếp cận này phù hợp với kiến trúc MLS hiện đại, trong đó MLS không bị ràng buộc vào duy nhất một mô hình triển khai tập trung, mà có thể được tích hợp vào nhiều kiến trúc hệ thống khác nhau, bao gồm cả môi trường P2P **rfc9750**. Từ đó, đề án đặt lại bài toán nghiên cứu theo hướng: thay thế chức năng điều phối thường do Delivery Service đảm nhiệm bằng một lớp điều phối phi tập trung ở tầng ứng dụng.

Đóng góp thứ hai là đề xuất cơ chế *Single-Writer per Epoch* cho nhóm MLS trên mạng P2P. Cơ chế này xác định một *Token Holder* duy nhất tại mỗi epoch bằng một quy tắc tất định dựa trên trạng thái nhóm, số epoch và tập thành viên hợp lệ. Nhờ vậy, trong điều kiện các nút có cùng góc nhìn về nhóm, toàn bộ hệ thống có thể tự tính ra cùng một nút chịu trách nhiệm tạo Commit mà không cần một vòng bỏ phiếu hoặc đồng thuận mạng bổ sung. Cơ chế này trực tiếp nhắm vào nguyên nhân cốt lõi gây phân nhánh trạng thái MLS trong môi trường P2P: nhiều thành viên cùng tạo Commit cạnh tranh tại cùng một epoch.

Đóng góp thứ ba là mô hình hóa vòng đời phân nhánh và phục hồi của nhóm MLS khi xảy ra *network partition*. Đề án phân biệt rõ hai lớp vấn đề: xung đột Commit trong điều kiện mạng liên thông và phân kỳ trạng thái khi mạng bị chia cắt vật lý hoặc logic. Với trường hợp thứ nhất, Single-Writer giúp giảm khả năng sinh concurrent commits. Với trường hợp thứ hai, đề án đề xuất quy trình phát hiện fork, lựa chọn nhánh tiếp tục và đưa các nút ở nhánh không được chọn gia nhập lại trạng thái MLS hợp lệ. Cách tiếp cận này thể hiện rõ định hướng thiết kế của đề án: ưu tiên tính sẵn sàng cục bộ trong môi trường P2P, nhưng vẫn bảo đảm hệ thống có cơ chế hội tụ trở lại sau khi kết nối được khôi phục.

0.4.2 Đóng góp về mặt hiện thực kỹ thuật

Về mặt kỹ thuật, đề án hiện thực một nguyên mẫu ứng dụng liên lạc nhóm bảo mật trên mạng ngang hàng, kết hợp Go cho lớp mạng và điều phối, Rust/OpenMLS cho lớp mật mã, SQLite cho lưu trữ cục bộ và gRPC cho giao tiếp liên tiến trình. Thiết kế này giúp hệ thống tận dụng được thư viện MLS chuẩn thay vì tự xây dựng lại các thành phần mật mã phức tạp, đồng thời vẫn giữ được khả năng tùy biến ở tầng điều phối để xử lý các đặc thù của môi trường P2P như mất kết nối, thông điệp đến sai thứ tự, đồng bộ ngoại tuyến và phân mảnh mạng.

Một đóng góp kỹ thuật quan trọng khác là kiến trúc *stateful Rust sidecar*. Trong kiến trúc này, Rust sidecar duy trì các đối tượng `MlsGroup` đang hoạt động trong bộ nhớ, trong khi Go Host chỉ gửi định danh nhóm và dữ liệu cần xử lý qua gRPC. Cách tổ chức này tránh việc khởi tạo lại ngữ cảnh MLS hoặc truyền toàn bộ trạng thái cây mật mã qua IPC trong mỗi thao tác. Nhờ đó, hệ thống giảm chi phí serialization, giảm độ trễ giữa hai tiến trình và phù hợp hơn với các chuỗi thao tác liên tiếp như tạo Proposal, tạo Commit, xử lý Commit, mã hóa và giải mã thông điệp.

Bên cạnh lõi điều phối, đề án cũng tích hợp các chức năng ứng dụng cần thiết để kiểm chứng giải pháp trong kịch bản thực tế, bao gồm phát hiện thiết bị trong mạng nội bộ, truyền thông điệp qua mạng ngang hàng, đồng bộ khi thiết bị quay trở lại sau thời gian offline, lưu trữ cục bộ lịch sử hội thoại và truyền tệp đã mã hóa. Các thành phần này cho phép đánh giá giao thức không chỉ ở mức mô hình lý thuyết, mà còn trong một nguyên mẫu hệ thống hoàn chỉnh, nơi các vấn đề của mạng P2P và trạng thái MLS xuất hiện đồng thời.

Tổng hợp lại, đóng góp chính của đề án nằm ở việc kết hợp một lõi mật mã nhóm đã được chuẩn hóa với một lớp điều phối phi tập trung được thiết kế riêng cho môi trường P2P. Cách tiếp cận này giữ nguyên các bảo đảm mật mã của MLS, đồng thời bổ sung các cơ chế cần thiết để hệ thống có thể vận hành trong điều kiện không có Delivery Service trung tâm.

0.5 Bố cục đề án

Phần còn lại của báo cáo được tổ chức như sau.

Chương 2 trình bày nền tảng lý thuyết của đề tài, bao gồm mô hình mã hóa đầu cuối, MLS, TreeKEM, Delivery Service, các vấn đề về Commit trong MLS, mạng ngang hàng và các cơ chế liên quan đến đồng bộ trong hệ phân tán.

Chương 3 mô tả chi tiết phương pháp đề xuất, bao gồm kiến trúc Go – Rust sidecar, mô hình trạng thái nhóm, cơ chế Single-Writer theo epoch, kiểm tra epoch,

phát hiện phân nhánh và phục hồi sau network partition.

Chương 4 phân tích lý thuyết về tính đúng đắn và các giới hạn của giải pháp, tập trung vào các thuộc tính như nhất quán trạng thái, tính sẵn sàng khi phân mảnh mạng, phạm vi bảo toàn FS/PCS và độ phức tạp tính toán, bảng thông của các thao tác cập nhật nhóm **rfc9420**.

Chương 5 trình bày đánh giá thực nghiệm của nguyên mẫu hệ thống. Các thí nghiệm tập trung vào độ trễ tạo và xử lý Commit, chi phí cập nhật thành viên, thời gian đồng bộ sau khi thiết bị offline quay trở lại, ảnh hưởng của số lượng thành viên đến hiệu năng và so sánh với các baseline phù hợp.

Cuối cùng, Chương 6 tổng kết kết quả đạt được, chỉ ra các hạn chế còn tồn tại và đề xuất các hướng phát triển tiếp theo, bao gồm mở rộng mô hình đe dọa, tăng cường chống thành viên độc hại, đánh giá trên mạng diện rộng và cải thiện cơ chế phục hồi phân nhánh.